

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..47

GET STARTED

 Copy as Markdown

Get started with testing

Serverpod generates test tools that let you call your endpoints from a test the same way your app calls them, as ordinary Dart functions. Behind that call the tools run a real server against a real database, so a test exercises your endpoint, your models, and your queries together.

That means no separate server to start, no HTTP client to write, and no mocked database. It also means these are integration tests: they check your code end to end rather than one class in isolation. Plain unit tests still work as they always do, and they need none of this.

Two things follow from that:

- **Tests stay independent.** Anything a test writes to the database is rolled back once that test finishes, so tests never see each other's data and you do not clean up after yourself. Each `withServerpod` group also runs against its own temporary database, which is dropped when the group finishes.
- **Each test sets its own scene.** A session builder decides what the server state looks like for a call, such as who is signed in.

Set up

New projects are ready to test. If you have

`test/integration/test_tools/serverpod_test_tools.dart` in your server package, skip to [writing a test](#).

INFO

Projects created with Serverpod 2.1 or earlier need a few one-time setup steps first. See [Upgrade to 2.2](#).

The test tools are generated along with the rest of your code. With `serverpod start` running, regenerates them. Outside a session, run `serverpod generate` from your server directory.



Ask AI

Either way you get `test/integration/test_tools/serverpod_test_tools.dart`. The `test/integration` folder keeps these apart from your unit tests, which makes a project easier to navigate. To generate the file somewhere else, change `server_test_tools_path` in `config/generator.yaml`.

Write a test

Import the generated file rather than the `serverpod_test` package: it carries the helpers and your endpoints together.

```
import 'package:test/test.dart';

import 'test_tools/serverpod_test_tools.dart';

void main() {
  withServerpod('Given Greeting endpoint', (sessionBuilder, endpoints) {
    test('when calling `hello` then it returns a greeting', () async {
      final greeting = await endpoints.greeting.hello(sessionBuilder, 'Bob');
      expect(greeting.message, 'Hello Bob');
    });
  });
}
```

The `withServerpod` callback gives you the two things every test uses. The `sessionBuilder` describes the server state a call runs under, and `endpoints` holds your endpoints.

Run it:

```
dart test
```

New projects set `dataPath` under `database` in `config/test.yaml`, which makes the test server start and manage its own PostgreSQL. There is nothing to launch first. If your `test.yaml` connects to a PostgreSQL server without a `dataPath`, that server has to be running. Each test group gets its own database on it.

Next

- [Writing tests](#): the session builder, authentication, and seeding data.
- [Configuration](#): the options `withServerpod` accepts.
- [Advanced examples](#): streams, future calls, and business logic.
- [Best practices](#): conventions worth following.
- [Load testing](#): measuring a live server under concurrent traffic.